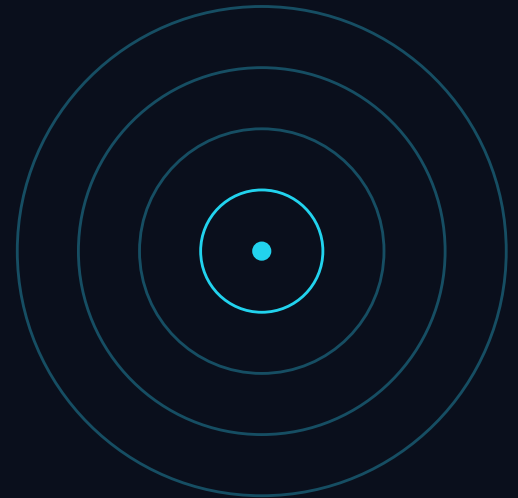


AI PORTFOLIO

# RAG-Assisted AI NOC Operator



*A 5-agent AIOps workflow that turns alarm floods into governed, runbook-cited actions: correlation, classification, safety governance, ITSM ticketing, and post-incident learning.*

[Live Demo](#)

[Claude API](#)

[OpenAI API](#)

[github.com/danvzla/ai-noc-operator-rag](https://github.com/danvzla/ai-noc-operator-rag)

Daniel Mazzini · Principal Architect & Senior TPM

THE  
PROBLEM

# Alarm floods, ungrounded automation, and lossy handoffs



## Alarm noise buries the signal

A single upstream fault can fan out into 200+ correlated alarms across routing, transport, and customer-impact systems, obscuring the one root cause.



## AI can't prove its reasoning

Most AIOps demos reason from prompts alone. Operators won't trust an automated action that can't cite the procedure behind it.



## Handoff context is lost

When automation stops or a human is needed, the next shift inherits a vague escalation, no ticket, no rollback state, no defined next action.

## THE APPROACH

# Automate the probably-safe, escalate the rest

### Auto-remediate everything

Fast but reckless. One wrong action mid-outage widens the blast radius.

Unsafe

### Escalate everything

Safe but worthless. If automation never acts, it offloads nothing from the operator.

No value

### Governed autonomy

Act only when the action is high-confidence, reversible, bounded, and runbook-backed. Everything else routes to a human.

This project

## ARCHITECTURE

# A 5-agent pipeline — each stage consumes the prior stage's structured output



▲ **RAG layer:** retrieved runbooks are injected into Agents 02 + 03 at prompt-build time

# Correlation, then classification

01

## Alarm Correlator

Collapses the raw alarm flood into a single probable root cause and a handful of correlated incident groups.

*IN: raw alarms → OUT: rootCause, groups, actionableCount*

02

## Incident Classifier

Scores each incident by confidence and severity, then recommends auto-remediation or human escalation — grounded in retrieved runbooks and required to cite them.

*IN: correlated incidents → OUT: confidence, action, runbook citation*

*Separation of concerns from the outset: correlation reasons about cause; classification reasons about response.*

## Governance is a separate step — confidence is not the same as safety

### The eligibility gate

An action is approved only if ALL hold:

- confidence  $\geq 65$
- documented rollback exists
- bounded blast radius
- software-reversible (no dispatch)
- no regulated traffic impacted



### Why it's isolated

- A confident classification is not a safe action — governance checks blast radius, rollback, and compliance separately
- The gate criteria live in a retrievable runbook (RB-GOV-001), not buried in a prompt — policy changes by editing a document
- Only “approve” reaches simulated execution; review and block route to ticketing

*Physical repairs, regulated traffic, and low-confidence calls are blocked or reviewed — never silently auto-fixed.*

# Ticketing, then closing the loop

04

## Ticketing & Shift Workflow

Generates ServiceNow/Remedy tickets for every escalated, reviewed, or blocked incident — with priority, assignment group, SLA, and the exact next-shift action.

*IN: governed incidents → OUT: ticketId, priority, owner, SLA*

05

## Post-Incident Learner

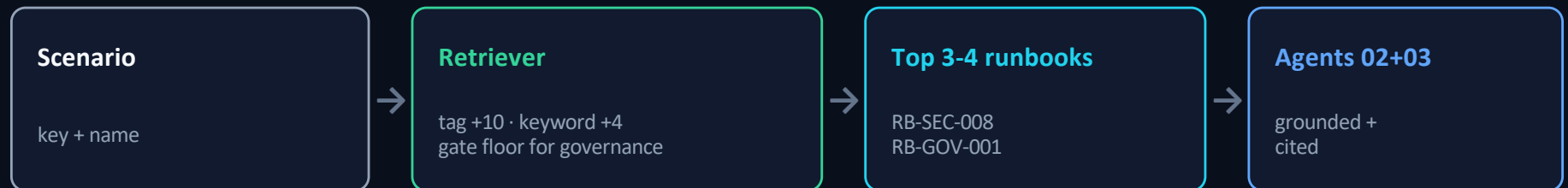
Produces a post-mortem and proposes runbook amendments where gaps surfaced — routed to human review, never auto-applied to the library.

*IN: full pipeline + outcome log → OUT: postmortem, proposed RB update*

*Human-in-the-loop extends to knowledge change: the same governance principle that gates actions also gates edits to the runbook library.*

## Retrieval grounds every decision in a cited runbook

Before Agents 02 and 03 run, the retriever pulls the runbooks matching the active scenario and injects them into the prompt. Agents must cite the runbook ID grounding each decision — turning “the model said so” into “per RB-SEC-008.”



**Deliberate scope:** tag/keyword retrieval, not vector search — the correct choice for a 14-runbook corpus. The retrieve() interface is shaped so a vector backend can drop in without touching agent code.



SAMPLE  
OUTPUT

# Every decision carries its citation

## Agent 02 — Classifier

**INC-2201 · conf 88 · AUTO**

*“per RB-SEC-008 step 4: stagger tunnel re-establishment to avoid thundering-herd”*

**INC-2204 · conf 58 · ESCALATE**

*“cause ambiguous — below RB-GOV-001 confidence floor”*

## Agent 03 — Governance

**INC-2201 → APPROVE**

*“rollback documented in RB-SEC-008; batched execution bounds blast radius”*

**INC-2207 → BLOCK**

*“blocked per RB-SEC-008: all-branch failover can overload secondary PoP — autoRemediationSafe: NO”*

WHAT THIS  
DEMONSTRATES

## The workflow at a glance

5

### Chained agents

correlate · classify · govern · ticket · learn

14

### Operational runbooks

6 domains, retrieval-indexed

6

### Scenarios

fiber · DDoS · BGP · DNS · 5G RAN · SASE

100%

### Cited decisions

every auto/block references a runbook

3-way

### Governance gate

approve · review · block

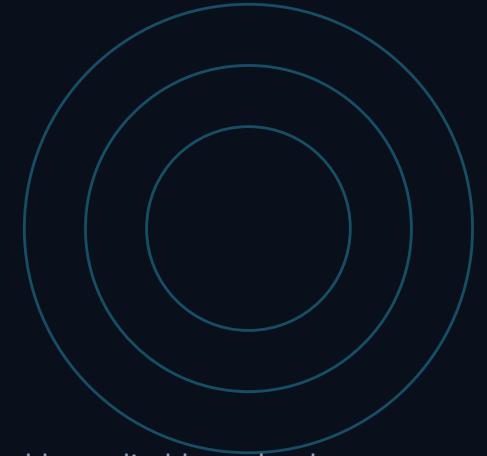
2

### Transparency panels

live prompts + retrieved runbooks

TRY IT

# Run a full incident in ~30 seconds — no API key



## Governed by design

Risky, uncertain, and regulated actions always route to a human. The gate criteria are a retrievable, auditable runbook.



## Fully traceable

Every auto and block decision cites the runbook behind it — no black-box reasoning.



## Grounded in real operations

Runbooks and scenarios drawn from two decades of network operations and telco delivery.

[Live demo](#) · [Demo, Claude, or OpenAI mode](#)

[github.com/danvzla/ai-noc-operator-rag](https://github.com/danvzla/ai-noc-operator-rag)